VPC (Virtual Private Cloud)

NAT GATEWAY (Network Address Translation)

1. you can use network translation getway to enable instances in a private subnet to connect to the internet or other aws services. but prevent the internet from initialling a connection with those instances.
2. you are charged for
    1. NAT Gateway hourly usage
    2. Data processing (per GB)
    3. Additional data transfer charges from Amazon EC2 may also apply.
3. NAT Gateway must be created in a public subnet so it can communicate with the internet.
4. While creating a NAT Gateway, you must associate an Elastic IP address (public IP) with it.
5. Instances in the private subnet do NOT require a public IP address to access the internet through NAT Gateway.
6. after you have created a nat gateway
    1. Update the route table of private subnets
    2. Add a route: 0.0.0.0/0 → NAT Gateway
    3. This allows outbound internet access from private instances.
7. When we Delete a NAT gateway
    1. The associated Elastic IP is disassociated
    2. But it is NOT automatically released from your account (you must release it manually to avoid charges)

BASTION-HOST SERVER / JUMP SERVER

1. A bastion host server acts as a gateway to a private network or server that is not directly accessible from the internet.
2. It provides a single-entry point for users who need to access specific resources on the private network.
3. All inbound traffic to the private network is routed through the bastion host server, which allows administrators to control and monitor access to the private network.
4. It enhances security by restricting access to sensitive systems and data to only authorized users.
5. It enables organizations to better control user activity and quickly detects and respond to potential security threats.

6. Using a bastion host server can reduce the risk of unauthorized access, data breaches, and other security incidents.
7. Overall, a bastion host server is an important tool for securing access to private networks and resources.

PRACTICAL ON NAT GATEWAY

1. **create a vpc** (10.0.0.0/16)
2. **create a subnet**
    a) 10.0.0.0/20 ----------> public subnet-1a
    b) 10.0.16.0/20 ----------> public subnet-1b
    c) 10.0.32.0/20 ----------> private subnet-1a
    d) 10.0.48.0/20 ----------> private subnet-1b
3. **create a internet gateway and attach to a vpc**
4. **create a nat-gateway**
    1. get public subnet-1a
    2. associates elastic ip address
5. **create a route-table**
    a) Public-route-table
        1. add public subnets
        2. attach Internet gateway
    b) Private-route-table
        1. add private subnets
        2. attach nat-getway
6. **create a instances**
    1. bastion-host-server (public server)
    2. AppServer (private server)
7. **login with bastion-host to get private server access**
    1. Save private-server key on bastion-host server
       vim private-server.pem
    2. chmod 600 private-server.pem
    3. ssh -i private-server.pem ubuntu@(private ip of private server)
    4. ping 8.8.8.8

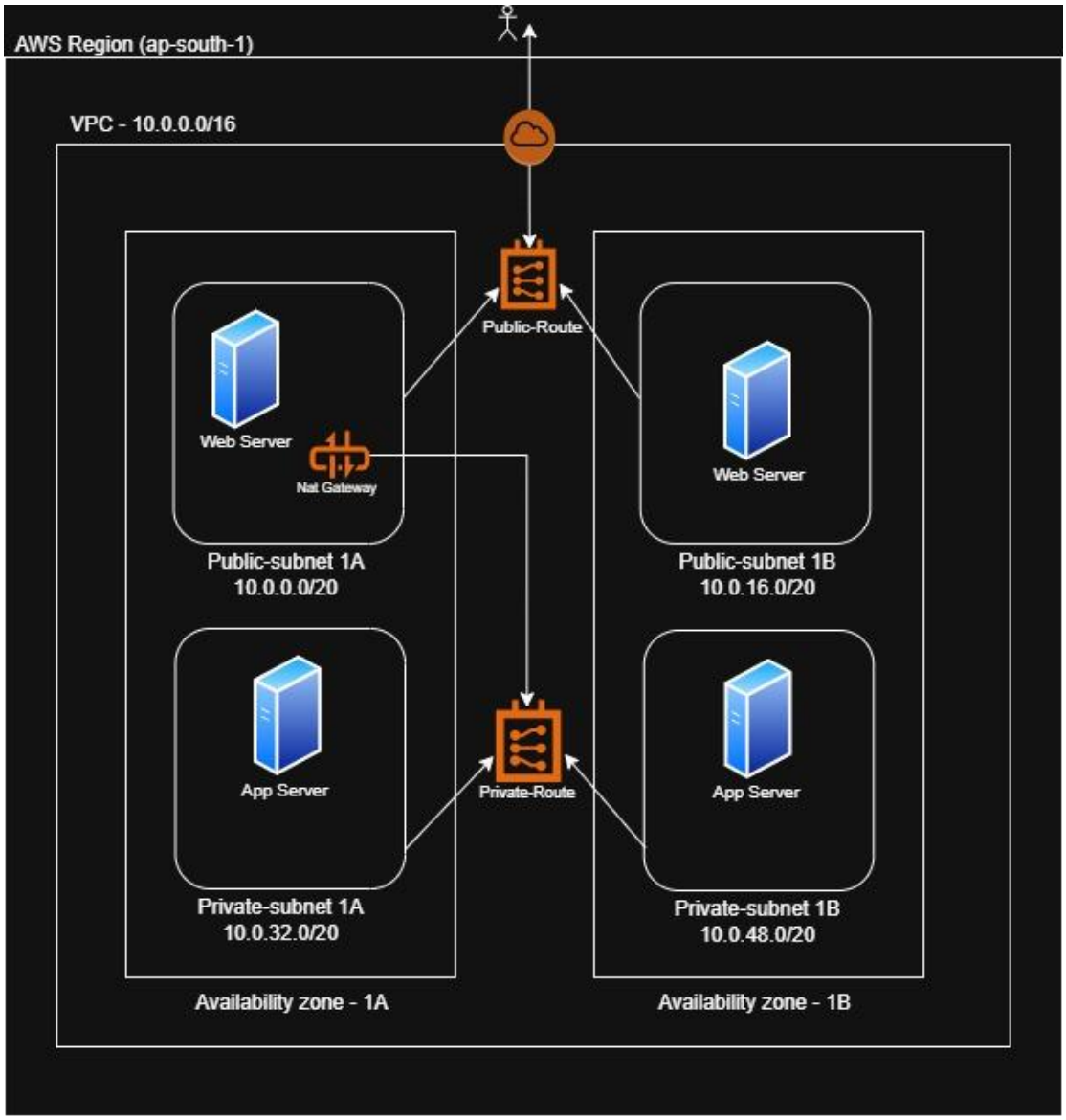
HOW TO DELETE A VPC

1. Delete EC2 instances
2. Delete NAT Gateway
3. Release Elastic IP

4. Detach & Delete Internet Gateway
5. Delete Route Tables
6. Delete Subnets
7. Delete VPC

TASK

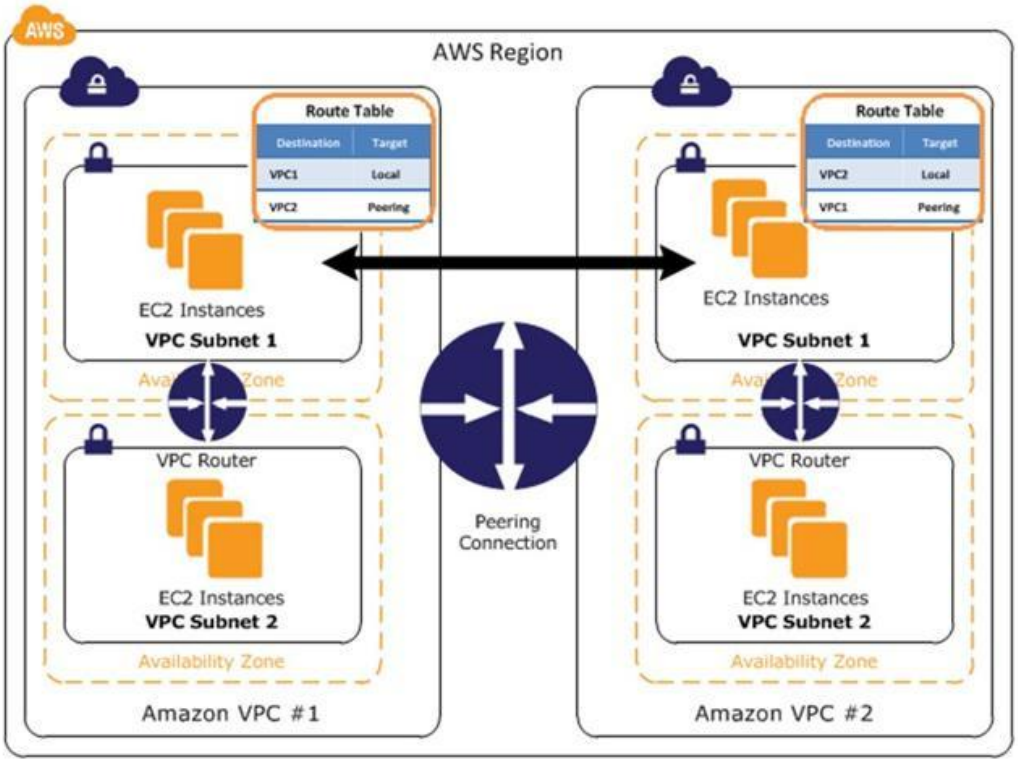
1. Perform Nat gateway practical on Windows server



VPC PEERING CONNECTION

1. a vpc peering connection is a networking connection between two vpc that enables you to route traffic between them using private IPv4 addresses or IPv6 address.
2. instance in either vpc can communicate with each other as if they are within the same network.
3. you can create a vpc peering connection between you own vpc or with a vpc in another aws account and the vpc can be in different region.

PRACTICAL ON VPC-PEERING CONNECTION



VPC-1 Configuration

1.Create A Vpc

- name ------> vpc-1
- cidr ---------> 10.0.0.0/16

2.Create A Subnets

- Subnet-1A ------> 10.0.1.0/24
- Subnet-1B ------> 10.0.2.0/24

3.create a internet gateway and attach to the vpc

4.Create Route Table (RT-VPC-1)

Attach internet gateway

Associates' subnets

- Subnet-1A
- Subnet-1B

5.Launch EC2

1. Name ----------> EC2-VPC1
2. Ami ----------> ubuntu
3. Instance type -> t3.micro
4. vpc ----------> vpc-1
5. subnet ---------> Subnet-1A
6. sg ----------> all traffic (allow) / ICMP
7. create a instance

VPC-2 Configuration

1.Create A Vpc

- name ------> vpc-2
- cidr ---------> 192.168.0.0/16

2.Create A Subnets

- Subnet-2A ------> 192.168.1.0/24
- Subnet-2B ------> 192.168.2.0/24

3.create a internet gateway and attach to the vpc

4.Create Route Table (RT-VPC-2)

Attach internet gateway

Associates' subnets

- Subnet-2A
- Subnet-2B

5.Launch EC2

1. Name ----------> EC2-VPC2
2. Ami ----------> ubuntu
3. Instance type -> t3.micro
4. vpc ----------> vpc-2
5. subnet ---------> Subnet-2A
6. sg ----------> all traffic (allow) / ICMP
7. create a instance

CREATE PEERING CONNECTION

1. create a peering connection

1. Name: VPC1-VPC2
2. Requester: VPC-1

3. Accepter: VPC-2
4. Accept Request

2.Update Route Tables
In VPC-1 (RT-VPC-1)
1. Destination: 192.168.0.0/16
2. Target: Peering Connection

In VPC-2 (RT-VPC-2)
1. Destination: 10.0.0.0/16
2. Target: Peering Connection

UPDATE SECURITY GROUPS

EC2-VPC1
1. Allow All TCP from 192.168.0.0/16

EC2-VPC2
2. Allow All TCP from 10.0.0.0/16

TEST CONNECTION

1.From EC2-VPC1
ping <Private-IP-of-EC2-VPC2>

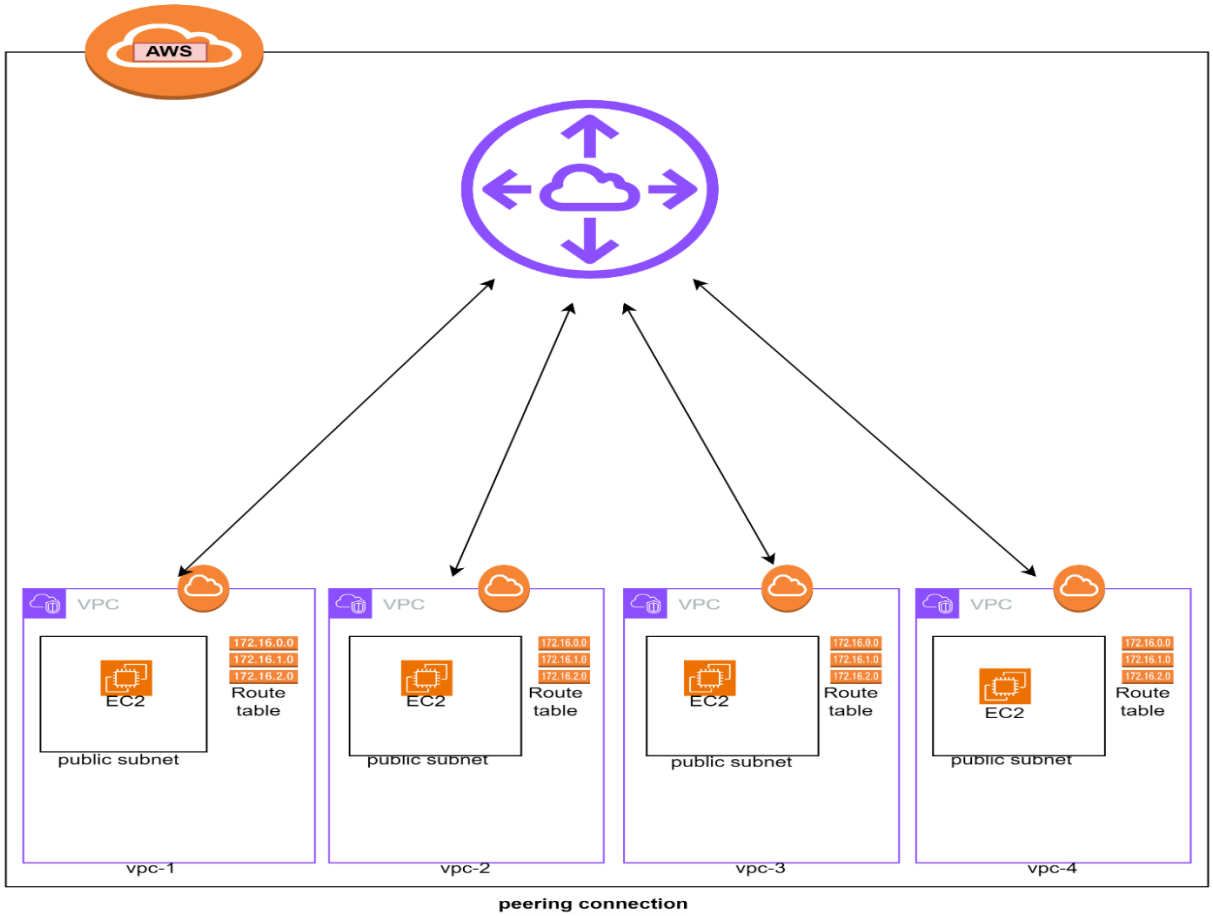
2.From EC2-VPC2
ping <Private-IP-of-EC2-VPC1>

Note:
1. if u just want to check whether VCP-1 connect to VCP-2 or not then IGW not required.
2. But if u ping the server, then u must attach ICMP protocol in both sg

---

CREATE A TRANSIT PEERING CONNECTION

Transit peering is when one network allows traffic to pass through it to connect two other networks that are not directly connected.



PRACTICAL ON TRANSIT GATEWAY

1.CREATE A VPC1 / VPC2 / VPC3 / VPC4

vpc1

1.create a vpc-1 ----------12.0.0.0/16
2.create a subnet--(subnet-vpc-1)-------12.0.1.0/24
3.create a IGW and attach to the vpc-1
4.create a route table--(rt-vpc-1)
associate---subnet-vpc-1----IGW
5.create a instance
1.name---------ec2-vpc-1
2.machine------amazon-linux
3.type---------t2.micro
4.key----------ec2-vpc-1 key
5.sg ----------ssh / http / allow TCP 12/13/14/15

6.user-data

```
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd.service
sudo echo "<h1> this is my 1st vpc </h1>" > /var/www/html/index.html
sudo systemctl restart httpd.service
sudo systemctl enable httpd.service
```

vpc2

1.create a vpc-2 ----------13.0.0.0/16
2.create a subnet--(subnet-vpc-2)-------13.0.1.0/24
3.create a IGW and attach to the vpc-2
4.create a route table--(rt-vpc-2)
associate---subnet-vpc-2----IGW
5.create a instance
1.name---------ec2-vpc-2
2.machine------amazon-linux
3.type---------t2.micro
4.key----------ec2-vpc-2 key
5.sg ----------ssh / http / allow TCP 12/13/14/15

6.user-data

```
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd.service
sudo echo "<h1> this is my 2nd vpc </h1>" > /var/www/html/index.html
sudo systemctl restart httpd.service
sudo systemctl enable httpd.service
```

vpc3

1.create a vpc-3 ----------14.0.0.0/16
2.create a subnet--(subnet-vpc-3)-------14.0.1.0/24
3.create a IGW and attach to the vpc-3
4.create a route table--(rt-vpc-3)
associate---subnet-vpc-3----IGW
5.create a instance
1.name---------ec2-vpc-3
2.machine------amazon-linux
3.type---------t2.micro

4.key----------ec2-vpc-3 key
5.sg ----------ssh / http / allow TCP 12/13/14/15
6.user-data
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd.service
sudo echo "<h1> this is my 3rd vpc </h1>" > /var/www/html/index.html
sudo systemctl restart httpd.service
sudo systemctl enable httpd.service

vpc4

1.create a vpc-4 ----------15.0.0.0/16
2.create a subnet--(subnet-vpc-4)-------15.0.1.0/24
3.create a IGW and attach to the vpc-4
4.create a route table--(rt-vpc-4)
associate---subnet-vpc-4----IGW
5.create a instance
1.name---------ec2-vpc-4
2.machine------amazon-linux
3.type---------t2.micro
4.key----------ec2-vpc-4 key
5.sg ----------ssh / http / allow TCP 12/13/14/15
6.user-data
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd.service
sudo echo "<h1> this is my 4th vpc </h1>" > /var/www/html/index.html
sudo systemctl restart httpd.service
sudo systemctl enable httpd.service

2.CREATE A TRANSIT GETWAY

1. name-------tg-for-all-vpcs
2. create a transit getway

3.CREATE TRANSIT GETWAY ATTACHMENTS-FOR-VPC-1

1. name----------tg-attachment-vpc-1
2. tg ID--------- tg-for-all-vpcs
3. type----------vpc
4. vpc ID--------(vpc-1)
5. create a tg attachments

4.CREATE TRANSIT GETWAY ATTACHMENTS-FOR-VPC-2

1.name----------tg-attachment-vpc-2
2.tg ID--------- tg-for-all-vpcs
3.type----------vpc
4.vpc ID--------(vpc-2)
5.create a tg attachments

5.CREATE TRANSIT GETWAY ATTACHMENTS-FOR-VPC-3

1.name----------tg-attachment-vpc-3
2.tg ID-------- tg-for-all-vpcs
3.type---------vpc
4.vpc ID-------(vpc-3)
5.create a tg attachments

6.CREATE TRANSIT GETWAY ATTACHMENTS-FOR-VPC-4

1.name----------tg-attachment-vpc-4
2.tg ID-------- tg-for-all-vpcs
3.type---------vpc
4.vpc ID-------(vpc-4)
5.create a tg attachments

6.UPDATE TRANSIT GATEWAY ROUTE TABLE

1. go to TGW → route tables
2. edit routes
3. add:
    1) 12.0.0.0/16 → attachment-vpc-1
    2) 13.0.0.0/16 → attachment-vpc-2
    3) 14.0.0.0/16 → attachment-vpc-3
    4) 15.0.0.0/16 → attachment-vpc-4

7.ENABLE PROPAGATION

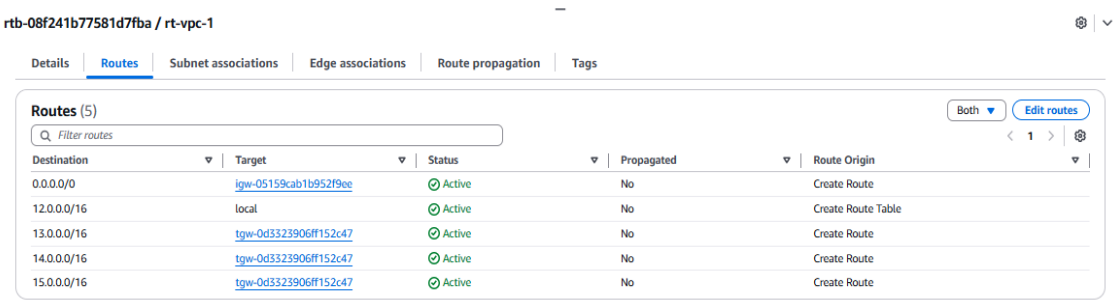
1. go to propagation tab
2. enable propagation for all attachments

Note:
Transit gateway route table and enable propagation if suppose u are not enable while creating the transit gateway

7.UPDATES THE ROUTS OF VPC-1 -> In vpc-1 we can add (vpc2-vpc3-vpc4)

1.add route------13.0.0.0/16 ----------transit getway---------save
2.add route------14.0.0.0/16----------transit getway---------save

3.add route------15.0.0.0/16----------transit getway---------save



8.UPDATE THE ROUTS OF VPC-2 -> In vpc-2 we can add (vpc1-vpc3-vpc4)

1.add route------12.0.0.0/16 ----------transit getway----------save
2.add route------14.0.0.0/16----------transit getway----------save
3.add route------15.0.0.0/16----------transit getway----------save

9.UPDATE THE ROUTS OF VPC-3 -> In vpc-3 we can add (vpc1-vpc2-vpc4)

1.add route------12.0.0.0/16 ----------transit getway----------save
2.add route------13.0.0.0/16----------transit getway----------save
3.add route------15.0.0.0/16----------transit getway----------save

10.UPDATE THE ROUTS OF VPC-4 -> In vpc-4 we can add (vpc1-vpc2-vpc3)

1.add route------12.0.0.0/16 ----------transit getway----------save
2.add route------13.0.0.0/16----------transit getway----------save
3.add route------14.0.0.0/16----------transit getway----------save

11.TEST THE INSTANCES

1. connect ec2-vpc-1
   ssh -i vpc-1.pem ec2-user@<public-ip>

Test connectivity

curl http://<private-ip-vpc-2>
curl http://<private-ip-vpc-3>
curl http://<private-ip-vpc-4>

```
[root@ip-12-0-1-154 ~]# ls
[root@ip-12-0-1-154 ~]#
[root@ip-12-0-1-154 ~]# curl localhost
<h1> These is my 1st vpc </h1>
[root@ip-12-0-1-154 ~]#
[root@ip-12-0-1-154 ~]# curl 13.0.1.172
<h1> This is my vpc-2 </h1>
[root@ip-12-0-1-154 ~]#
[root@ip-12-0-1-154 ~]# curl 14.0.0.195
<h1> These is my vpc-3 </h1>
[root@ip-12-0-1-154 ~]#
[root@ip-12-0-1-154 ~]#
```

VPC ENDPOINT

A vpc endpoint enables you to privately connect your vpc to supported aws services instance in your vpc do not require public IP addresses to communicate with resources in the service.

There are two types of vpc endpoints

1. interface
2. getway

Difference between interface and getway

1. Interface endpoints use an elastic network interface (ENI) as an entry point for traffic to the AWS service, while gateway endpoints use a VPC endpoint gateway.
2. Interface endpoints are used for services that have a private IP address within the VPC, such as Amazon S3 and Amazon DynamoDB, while gateway endpoints are used for services that have a public IP address, such as Amazon Kinesis and Amazon CloudWatch.
3. Interface endpoints are highly available and scalable, with support for multiple ENIs and multiple IP addresses per ENI, while gateway endpoints are typically limited to a single endpoint per VPC.
4. Interface endpoints are associated with a subnet and require a route table entry to direct traffic to the endpoint, while gateway endpoints are associated with a VPC and do not require any additional configuration.
5. Interface endpoints support endpoint policies, which allow you to control access to the endpoint, while gateway endpoints do not support endpoint policies.

PRACTICAL ON VPC ENDPOINT

1.create a vpc

1. name ---------> my-vpc
2. CIDR ----------> (10.0.0.0/16)

2.create a subnet

1. public subnet (10.0.0.0/24)
2. private subnet (10.0.1.0/24)

3.create a internet getway / attach to a vpc

4.create a route table

1.public-routetable
   1.add subnet ----> public subnet
   2.add route ------> internet getway
2.private-routetable
   1.add subnet -----> private subnet

5.create a endpoint

1. name --------> my-s3-endpoint
2. aws services
3. service add ---> amazon s3
4. select vpc -----> my-vpc
5. route table ---> private-routetable
6. create a endpoint

6.create a s3 bucket

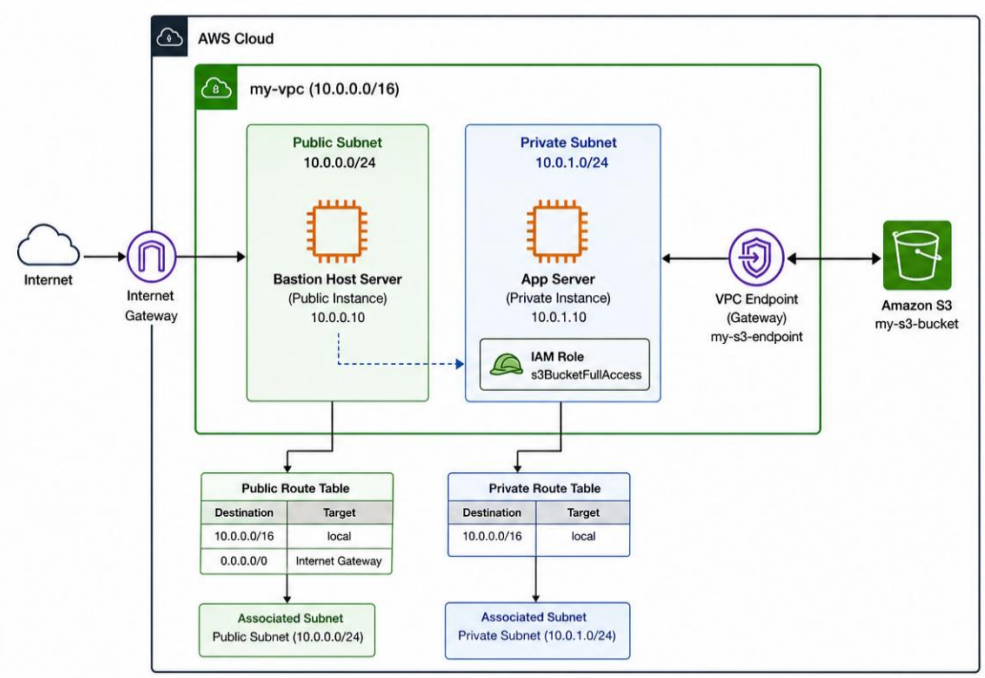
1. my-s3-bucket

7.create a instances

1. bastion-host-server (public server)
2. app-server (private server)

7.login on bastion-host-sever to get access of app server

1. vim private.pem
2. chmod 600 private.pem
3. ls -l private.pem
4. ssh -i private.pem ec2-user@(private ip of private server)
5. check aws cli
   aws configure
6. attach role to the private instance (s3BucketFullAccess)
7. aws s3 ls
   my-s3-bucket



PRACTICAL ON Network ACL

1.create a vpc

1.name --------> my-vpc
2.CIDR --------> 10.0.0.0/16

2.create a subnet

public subnet---10.0.0.0/24

3.create a internet getway and attach to a vpc

4.create a route table

Public route ----> Attach internet gateway & public subnet

5.creat a network acl

1.associate a subnet -----> public subnet
2.add inbound permission
100---ssh
3.add outbound permission
100---http
200---https

6.create a ec2 instance

1.name ---------> myserver
2.machine -----> ubuntu
3.sg ----------> ssh (22)
4.create a instance

7.access the server

{ you can not access the server because u have only allow traffic in inbound in nacl you must assign permission in the outbound as well }

outbound traffic-----> 250—custom TCP----> 1024-65535

8.Then you get the ssh of these ec2 server
